

GITAM School of Business

Visakhapatnam | Hyderabad | Bengaluru

Customer Review Sentiment Analysis

Classifying Amazon app reviews and identifying common customer concerns using
Natural Language Processing

Capstone Project (Assessment A2)

24INFS7101 | NLP and Text Analytics in Business

Item	Details
Submitted to	Siva Veludurthi
Programme	MBA, GITAM School of Business, Hyderabad
Team members	Patlolla Vamshinath Reddy Deepika Rama Krishna Arpitha Abhinaya Krishna
Academic year	2026-2027
Submission date	01-10-2026

1. Executive Summary

Online reviews are one of the richest sources of customer feedback a company has, but once they run into the thousands nobody can read them all. This project applies Natural Language Processing (NLP) to **19,996 Amazon app reviews** to do two things: automatically classify each review as positive or negative, and find out what customers are actually complaining about.

We cleaned the text, converted it into numerical features using Bag of Words and TF-IDF (unigrams and bigrams), and compared four classifiers on a held-out test set. The best model, **Logistic Regression on Bag of Words, reached 90.2% accuracy** and correctly caught 85.7% of the negative reviews (negative-class F1 of 0.81). Because the data is imbalanced (76% positive), we report precision, recall and F1 alongside accuracy. A useful side result is that TF-IDF with Naive Bayes looked acceptable on accuracy (86.7%) but found only 48% of negative reviews, which is exactly the kind of failure accuracy hides.

To understand the complaints, we ran keyword analysis and Latent Dirichlet Allocation (LDA) topic modelling on the 4,766 negative reviews. Five themes emerged. The largest is technical: downloads, installation problems and apps that force-close, often on Kindle devices (26.6% of negative reviews). Next are a sense of wasted time and money (22%) and confusion about free versus paid versions (18.1%). We turn these into four practical recommendations in Section 7.

2. Introduction and Business Problem

2.1 Background

Businesses now receive customer opinions through review sites, app stores, emails and social media. This text is unstructured, so it does not fit neatly into rows and columns the way sales or inventory data does. That makes it hard to use in decisions, even though it often explains why customers stay or leave. NLP gives a way to turn this text into measurable signals such as sentiment scores and recurring themes.

2.2 Problem Statement

Businesses receive thousands of customer reviews, and reading them manually is slow, inconsistent and hard to scale. As a result, recurring customer concerns are often noticed late, which affects satisfaction and retention. This project applies NLP to a large set of Amazon app reviews to classify sentiment and to identify the most common customer concerns, so that management can prioritise product and service improvements.

2.3 Objectives

1. Clean and preprocess raw review text into a form suitable for modelling.
2. Classify reviews as positive or negative using Bag of Words and TF-IDF features with Naive Bayes and Logistic Regression, and compare the models fairly.
3. Identify the main complaint themes in negative reviews using keyword analysis and LDA topic modelling.

4. Present the results visually and convert them into data-backed business recommendations.

2.4 Scope

The problem statement originally mentioned positive, negative and neutral classes. The dataset we used carries only two labels (positive and negative), so the classification task is binary. We chose to stay with the labels provided rather than invent a neutral class that we could not validate. A neutral class is listed under future work.

2.5 Link to Course Outcomes

Outcome	How the project addresses it
CO2	Text preprocessing, Bag of Words and TF-IDF feature engineering (Sections 5.2 and 5.3).
CO3	Sentiment classification, keyword extraction and LDA topic modelling (Sections 5.4 to 6.5).
CO4	Application of NLP to a customer-experience problem, with a pipeline that can be automated (Section 7).
CO5	Recommendations, ethical considerations, limitations and future work (Sections 7 to 10).

3. Literature Review

Sentiment analysis, also called opinion mining, studies how opinions and attitudes are expressed in text (Liu, 2012; Pang & Lee, 2008). Early work by Pang, Lee and Vaithyanathan (2002) showed that standard machine-learning classifiers such as Naive Bayes and Support Vector Machines could classify movie reviews by sentiment, and that simple word-presence features performed well. That result is the reason bag-of-words style features are still a sensible baseline for review data today.

Two families of approach are commonly compared. Lexicon-based methods such as VADER score text using a dictionary of sentiment-bearing words and rules for negation and intensity, and were designed for short social-media text (Hutto & Gilbert, 2014). Supervised methods learn from labelled examples and usually adapt better to a specific domain, at the cost of needing labels. Our dataset comes with labels, so we use the supervised route.

For text representation, term frequency-inverse document frequency (TF-IDF) down-weights words that appear in many documents and gives more weight to distinctive ones (Salton & Buckley, 1988). For discovering themes without labels, Latent Dirichlet Allocation models each document as a mixture of topics and each topic as a distribution over words (Blei, Ng & Jordan, 2003). We use it on the negative reviews to summarise what customers complain about.

Finally, review datasets are rarely balanced, and plain accuracy can be misleading when one class dominates (He & Garcia, 2009). We therefore evaluate with precision, recall and F1 as well, and we pay particular attention to the minority (negative) class, since that is the class a business most needs to detect. More recent transformer models such as BERT (Devlin et al., 2019) generally improve accuracy but are heavier to run and harder to explain; we discuss them as future work.

4. Data Description

4.1 Source and Structure

The dataset is a CSV file (amazon.csv) with two columns: **Text**, the free-text review, and **label**, where 1 marks a positive review and 0 a negative one. The review text refers to Kindle devices and mobile games, which is consistent with app-store reviews from the Amazon Appstore. The file does not document its original source or how the labels were assigned. We assume they were derived from star ratings, and we treat them as ground truth for training and evaluation.

4.2 Data Quality Checks

Check	Result
Number of reviews	19,996
Missing values	None in either column
Duplicate review texts	None
Positive reviews (label 1)	15,230 (76%)
Negative reviews (label 0)	4,766 (24%)
Review length (words)	Mean 33.9, median 31, range 1 to 61
Mean length by class	Positive 34.2, negative 33.3

4.3 Exploratory Findings

Two features of the data shaped our approach. First, the classes are imbalanced, with roughly three positive reviews for every negative one (Figure 1). This is why we use stratified splitting, class weights and class-specific metrics. Second, reviews are short, with a mean of about 34 words and none longer than 61, and positive and negative reviews are almost the same length, so length carries no useful signal. Short text also means each word matters, which favours simple word-count features and makes negation handling important.

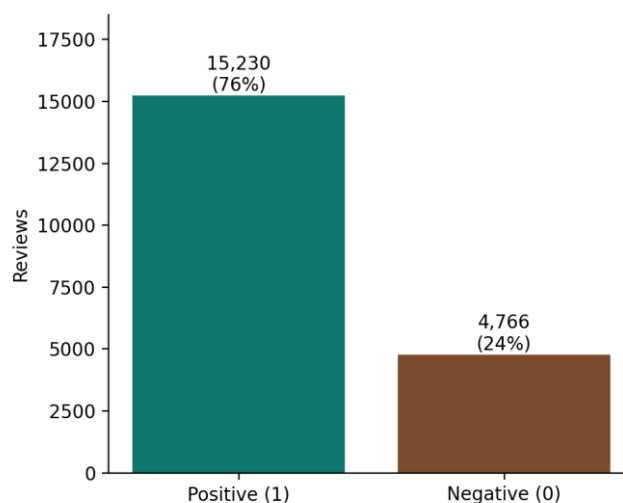


Figure 1: Distribution of reviews by sentiment label

5. Methodology

5.1 Overview

The workflow follows the standard NLP pipeline: clean the text, break it into tokens, filter out uninformative words, convert text into numeric features, train and compare models, and finally interpret the results. All work was done in Python using pandas, scikit-learn (Pedregosa et al., 2011) and matplotlib, with a fixed random seed of 42 so results can be reproduced.

5.2 Text Preprocessing

- **Decoding and lowercasing.** HTML entities such as " and ' were decoded, and all text was converted to lower case. Without this step, tokens like "apos" and "quot" appeared as false keywords in our first run.
- **Noise removal.** URLs, digits and punctuation were removed, and contractions ending in n't were expanded to "not" so that negation survives cleaning.
- **Tokenisation.** Text was split into word tokens, and single-character tokens were dropped.
- **Stopword removal.** We used the scikit-learn English stopwords list, but deliberately kept negation words (not, no, nor, never, cannot, without, against). Removing them would turn "not worth it" into "worth", reversing the meaning. This choice pays off in the results: "not", "not worth" and "not recommend" are among the strongest negative predictors.

We did not apply stemming or lemmatization in this version. Bigram features already capture many useful word combinations, and adding a lemmatizer is listed under future work.

5.3 Feature Engineering

We built two text representations, both using unigrams and bigrams and ignoring terms that appear in fewer than three reviews:

- **Bag of Words (BoW):** each review becomes a vector of raw word and word-pair counts.
- **TF-IDF:** the same vocabulary, but counts are scaled (sublinear term frequency) and weighted by how rare a term is across reviews, so distinctive words count for more than common ones.

5.4 Models

We trained four models, combining each representation with two classifiers:

- **Multinomial Naive Bayes** is a fast probabilistic baseline that suits word-count data.
- **Logistic Regression** is a linear classifier whose coefficients show which terms push a review toward positive or negative. We used balanced class weights so that the minority class is not ignored.

5.5 Evaluation Design

The data was split 80/20 into training (15,996 reviews) and test (4,000 reviews) sets, stratified so both keep the same 76/24 class ratio. Every model was fitted on the training set only and scored on the

untouched test set. Because the negative class is the one a business cares about, we report accuracy, precision, recall and F1 for the negative class, and macro F1 (the average of the two classes' F1 scores).

5.6 Theme Discovery

To understand why customers are unhappy, we analysed only the negative reviews. We first extracted top keywords using average TF-IDF, and then fitted an LDA model with five topics on word counts. Generic words such as "game", "app" and "play" were removed for this step because they appear in almost every review and would blur the topics. We assigned each review to its dominant topic and report each topic's share. The topic names in Section 6.4 are our own interpretation of the top words.

6. Results and Analysis

6.1 Model Comparison

Model	Accuracy	Neg. precision	Neg. recall	Neg. F1	Macro F1
BoW + Naive Bayes	0.897	0.760	0.833	0.795	0.863
TF-IDF + Naive Bayes	0.867	0.920	0.482	0.632	0.775
TF-IDF + Logistic Regression	0.899	0.751	0.866	0.804	0.868
BoW + Logistic Regression	0.902	0.761	0.857	0.806	0.870

Table 1: Test-set performance of the four models (negative = class 0)

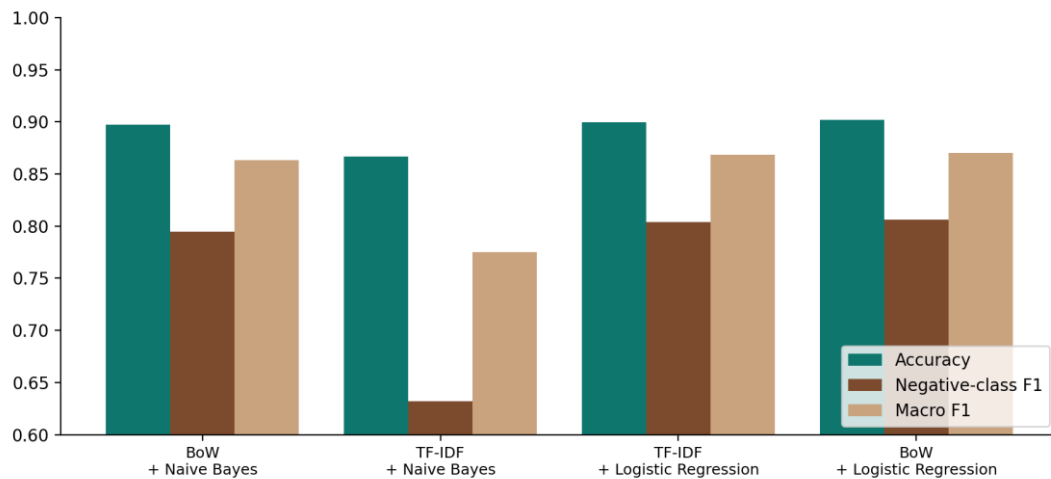


Figure 2: Accuracy, negative-class F1 and macro F1 by model

Bag of Words with Logistic Regression performed best, with accuracy of 90.2%, negative recall of 85.7% and macro F1 of 0.870. TF-IDF with Logistic Regression was very close (accuracy 90.0%), so the choice between the two representations matters little for this dataset. That is plausible for short reviews, where IDF weighting has less to work with.

The clearest lesson comes from **TF-IDF with Naive Bayes**. Its accuracy of 86.7% looks respectable, but its negative recall is only 48.2%: it misses more than half of the negative reviews because it leans

toward the majority class. A company relying on that model would overlook most unhappy customers. This is why we did not choose a model on accuracy alone.

6.2 Error Analysis

	Predicted negative	Predicted positive	Total
Actual negative	817	136	953
Actual positive	256	2,791	3,047

Table 2: Confusion matrix for Bag of Words + Logistic Regression (test set, 4,000 reviews)

The model correctly flagged 817 of 953 negative reviews and 2,791 of 3,047 positive reviews. For the positive class, precision is 95.4%, recall is 91.6% and F1 is 0.934. The main weakness is that 256 positive reviews were flagged as negative, which is what pulls negative precision down to 76.1%. Balanced class weights deliberately trade some precision for recall on the negative class. For a monitoring tool this is a sensible trade, since a human can quickly dismiss a false alarm but cannot act on a complaint that was never surfaced. Some errors are likely to come from mixed reviews (praise followed by a complaint) and sarcasm, which word-count models cannot read reliably.

6.3 Words That Drive Sentiment

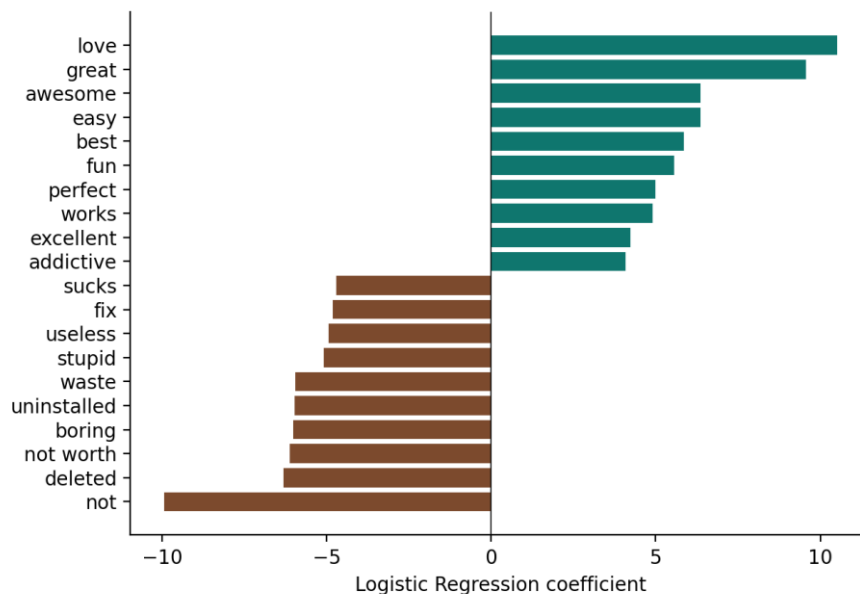


Figure 3: Terms with the largest Logistic Regression coefficients (TF-IDF model). Brown = negative, teal = positive

The strongest positive terms are love, great, awesome, easy, best, fun, perfect, works. The strongest negative terms are not, deleted, not worth, boring, uninstalled, waste, stupid, useless, fix. The results are intuitive, and they include two useful patterns for a manager: dissatisfied customers often report an action they took (deleted, uninstalled), and negation phrases (not worth, not recommend) rank among the most reliable negative markers. That supports our decision to keep negation words during preprocessing.

6.4 Complaint Themes from LDA

#	Theme (our label)	Share	Top words
1	Download, install and crash problems	26.6%	kindle, work, tried, time, screen, force, phone, download
2	Wasted time and money	22.0%	time, waste, stupid, buy, hate, money, dont, space
3	Free vs paid version and platform	18.1%	free, phone, amazon, version, android, need, no, use
4	Boring gameplay and Kindle compatibility	17.0%	kindle, bad, work, no, good, boring, say, reviews
5	Video and other-app expectations	16.2%	no, use, good, videos, thought, download, video, ca

Table 3: LDA topics on the 4,766 negative reviews. Share = reviews where the topic is dominant

The largest theme is technical. Words such as download, tried, screen, force and Kindle point to apps that would not install, would not run, or kept closing, often on Kindle devices. The second theme is about perceived value: waste, time, money, stupid and hate describe customers who felt the app was not worth their effort or spending. A third theme concerns the free versus paid versions and the platform (Android, Google, Amazon). Themes four and five are smaller: boredom and Kindle compatibility, and reviews comparing the app with video apps such as YouTube.

Topic models are exploratory, so these labels are our judgement, and some topics overlap (for example, *Kindle* appears in two of them). We therefore checked the themes against simple keyword counts, shown next.

6.5 Keyword Check

Keyword (stem)	Negative reviews	Share of negative
kindle	654	13.7%
download	629	13.2%
uninstall / delete	554	11.6%
free	474	9.9%
waste	426	8.9%
not work	376	7.9%
phone	332	7.0%
boring	218	4.6%
force (close)	170	3.6%

Table 4: How many of the 4,766 negative reviews mention each term (a review can mention several)

The counts agree with the LDA themes. About one negative review in seven mentions Kindle or downloading, one in nine mentions uninstalling or deleting the app, and roughly one in ten mentions the free version or calls the app a waste. Because a review can mention several of these, the percentages should not be added together.

7. Business Recommendations

Finding	Recommendation	Why it matters
Technical problems are the largest complaint theme (26.6%), especially download, install and force-close issues, often on Kindle.	Prioritise device-compatibility testing and crash fixes before each release, starting with Kindle devices.	These failures stop customers using the product at all, so they are likely the fastest way to improve ratings.
Free versus paid confusion appears as its own theme (18.1%).	State clearly on the store page what is free and what costs money.	Customers who feel misled by pricing leave negative reviews even when the product works.
"Waste of time and money" is a major theme (22%).	Improve onboarding and the first few minutes of use so that value is visible early.	Perceived poor value is an early-churn risk that better first impressions can address.
A simple model flags 86% of negative reviews automatically.	Use the classifier to route negative reviews to the support team and track theme shares release by release.	It replaces manual reading and lets management see whether fixes are working.

These recommendations come directly from the text evidence, but the dataset has no dates or product identifiers. We therefore cannot say which app or release caused each complaint, or whether complaints are rising over time. A production system should capture those fields.

8. Ethical Considerations

- **Privacy.** Reviews are public text and the dataset contains no names or contact details, but any real deployment should check terms of use and remove personal information before storing or sharing review data.
- **Bias and fairness.** The model learns from the words in one product category and one platform. It may misread slang, regional English, code-mixed text or sarcasm, and could under-serve those customers. Results should be checked on new data before they drive decisions.
- **Human oversight.** With about 76% precision on negative reviews, the model should support analysts rather than replace them, particularly where a decision affects a customer directly.
- **Transparency.** We chose linear models partly because their coefficients can be inspected and explained to non-technical stakeholders.

9. Limitations

- Only two sentiment labels are available, so mixed or neutral opinions are forced into positive or negative.
- The class split is imbalanced (76/24), and results come from a single train/test split rather than cross-validation.
- The origin of the dataset and the rule used to assign labels are not documented.

- No dates, product names or ratings are included, which rules out trend analysis and product-level comparison.
- Stemming or lemmatization and a lexicon baseline such as VADER were not part of this run, so we cannot yet compare our supervised models with a rule-based method.
- LDA topic labels are subjective, and results can change with the number of topics chosen.

10. Future Work

5. Add a VADER baseline and compare it with the supervised models on the same test set.
6. Add lemmatization and test whether it improves recall on the negative class.
7. Use cross-validation and tune hyperparameters, including the number of LDA topics.
8. Try a transformer model such as BERT (Devlin et al., 2019) and weigh the accuracy gain against cost and explainability.
9. Introduce a neutral class and aspect-based sentiment (for example, separate scores for stability, price and gameplay).
10. Connect the classifier to a dashboard that tracks sentiment and complaint themes over time.

11. Conclusion

This project shows that a straightforward NLP pipeline can turn 19,996 unstructured reviews into information a manager can act on. Bag of Words with Logistic Regression classified reviews with 90.2% accuracy and detected 86% of negative reviews, and a comparison with Naive Bayes showed why recall on the minority class must be checked. Topic modelling and keyword analysis then pointed to three clear priorities: technical reliability, perceived value, and clarity about free versus paid content. With dates and product identifiers added, the same approach could become a continuous customer-feedback monitoring tool.

References

- Blei, D. M., Ng, A. Y., & Jordan, M. I. (2003). Latent Dirichlet allocation. *Journal of Machine Learning Research*, 3, 993-1022.
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL-HLT*, 4171-4186.
- He, H., & Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9), 1263-1284.
- Hutto, C. J., & Gilbert, E. (2014). VADER: A parsimonious rule-based model for sentiment analysis of social media text. *Proceedings of the International AAAI Conference on Web and Social Media*.
- Kadre, S., Kadre, S., & Dey, S. *Mastering text analytics: A hands-on guide to natural language processing using Python*. Packt Publishing.
- Liu, B. (2012). *Sentiment analysis and opinion mining*. Morgan & Claypool.

- Pang, B., & Lee, L. (2008). Opinion mining and sentiment analysis. *Foundations and Trends in Information Retrieval*, 2(1-2), 1-135.
- Pang, B., Lee, L., & Vaithyanathan, S. (2002). Thumbs up? Sentiment classification using machine learning techniques. *Proceedings of EMNLP*, 79-86.
- Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5), 513-523.
- Sarkar, D. *Text analytics with Python: A practitioner's guide to natural language processing*. Apress.